

DRCAD: Dual-View Experts Routing and Counterfactual Generation for Explainable Time Series Anomaly Detection

Dawei Zhao¹, Member, IEEE, Haoran Li², Lijuan Xu², Zhen Wang³, Fellow, IEEE, and Haipeng Peng⁴

Abstract—Time series anomaly detection is critical in domains such as cybersecurity monitoring, network operations, and industrial control systems. Lately, unsupervised anomaly detection methods that utilize contrastive learning have shown promise. However, existing approaches often struggle to model high-dimensional temporal dependencies efficiently and rely on rigid feature-fusion schemes that can inadvertently amplify noise. These factors increase computational overhead and sensitivity to irrelevant signals, hindering the capture of salient patterns. Additionally, the explainability of anomalies detected by these mechanisms is often limited, restricting their application in traceable detection processes and an explicit decision-making basis. In this paper, we propose dual-view experts routing and counterfactual generation for explainable time series anomaly detection (DRCAD), a novel framework that detects anomalies within time series data while providing intuitive and actionable explanations for model predictions. DRCAD uses in-patch and patch-wise perspectives as input views for the contrastive learning model, employing a flattened attention mechanism with lightweight spatial projections and a Patch Mixture of Experts (MoE) layer for adaptive routing and information fusion. It identifies anomalies by expanding the discrepancy between normal and anomalous points in the representation space, subse-

quently outputting anomaly scores. These anomaly scores guide the generation of counterfactual samples, integrating feature change tendencies with normalized feature impacts to derive a feature importance ranking as the explanation. We evaluate DRCAD on six widely used datasets, observe state-of-the-art (SOTA) performance. Moreover, in the explainability evaluation on SWaT dataset, DRCAD achieves superior realism and sparsity in counterfactual generation compared to existing methods, with top-ranked features closely matching officially documented attack characteristics.

Index Terms—Time series, contrastive learning, anomaly detection, explainable, XAI, counterfactual explanation.

I. INTRODUCTION

ANOMALY detection is a critical task in many high-security domains. For example, in industrial control systems, experts aim to identify which sensor is malfunctioning or which device is under attack [1], [2]. In network security, anomaly detection is essential for identifying and mitigating threats such as intrusions, insider attacks, and performance issues [3], [4]. Insufficient detection of anomalous data can lead to severe consequences [5].

Detecting anomalies in real-world scenarios is challenging. In practice, anomalies constitute only a small fraction of the data and are generally weakly correlated with one another [6]. Current anomaly detection techniques primarily use statistical methods, traditional machine learning, and deep learning, with deep learning achieving significant success [7], [8]. Deep learning based anomaly detection methods are typically divided into prediction-based and reconstruction-based approaches [9]. In recent years, reconstruction-based methods have gained significant notice. These methods operate on the idea that a model, when trained exclusively on normal data patterns, will fail to accurately reconstruct anomalous data, thus identifying anomalies through reconstruction errors. However, these methods often face challenges such as large reconstruction errors, which can adversely affect model training and lead to false positives. To overcome these limitations, contrastive learning-based anomaly detection methods have emerged, offering a compelling alternative approach [6], [10], [11], [12], [13]. Contrastive learning focuses on learning representations by contrasting different views. Nevertheless, designing stable architectures for contrastive learning on time series is still challenging, and the effectiveness of such representations for anomaly detection warrants further investigation.

Received 15 August 2024; revised 22 March 2025 and 24 September 2025; accepted 28 November 2025. Date of publication 3 December 2025; date of current version 16 December 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3107300, in part by the National Natural Science Foundation of China under Grant 62172244, in part by Taishan Scholars Program under Grant tsqn202211210, in part by the “20 New Universities” Project of Jinan City under Grant 202333023 and Grant 202333045, in part by the Key Research and Development Program of Shandong under Grant 2025CXPT082, in part by Shandong Provincial Natural Science Foundation under Grant ZR2024MF250, and in part by the Pairing Plan Project of the School of Computer Science and Technology of Qilu University of Technology (Shandong Academy of Sciences) under Grant 2024JDJH12. The associate editor coordinating the review of this article and approving it for publication was Dr. Andrew Clark. (Corresponding authors: Dawei Zhao; Lijuan Xu.)

Dawei Zhao, Haoran Li, and Lijuan Xu are with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250316, China, and also with Shandong Provincial Key Laboratory of Industrial Network and Information System Security, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China (e-mail: zhaodw@sdas.org; haoranli0605@163.com; xulj@sdas.org).

Zhen Wang is with the School of Artificial Intelligence, Optics and Electronics (iOPEN), and the School of Mechanical Engineering, Northwestern Polytechnical University, Xi’an, Shaanxi 710072, China (e-mail: zhenwang0@gmail.com).

Haipeng Peng is with the Information Security Center, State Key Laboratory of Networking and Switching Technology, and the National Engineering Laboratory for Disaster Backup and Recovery, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: penghaipeng@bupt.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3639899

In addition, the opacity of internal representations in deep learning models is an urgent issue. For example, if a banking system labels someone as a defaulter, the individual may demand the basis for this judgment. Legal systems may only trust the model if it aligns with objective facts. Unfortunately, most time series anomaly detection models lack explainability, leaving users unaware of how decisions are made.

Several methods have been proposed to address the explainability problem. Feature importance techniques, such as SHAP [14] and LIME [15], provide insights into which features are most influential in a model's decision. However, these methods often fail to offer a clear understanding of how changes in the input would affect the output. In contrast, counterfactual explanations [16], [17], [18] seek the closest but different data point in a non-real space that would yield a different prediction under the model. These methods are commonly used in computer vision [19], where counterfactual images are generated and compared with the original images for explanation. However, applying counterfactual explanations to time series anomaly detection is challenging due to the unique temporal dependencies and intrinsic coupling of features in time series data, as well as the mixture of normal and anomalous samples, making it difficult for experts to discern the properties of generated samples [20], [21]. To handle the challenges above, after the model makes a prediction, we use an anomaly score-guided conditional variational autoencoder (CVAE) [22] to generate counterfactual samples. Unlike traditional VAEs, conditioning on the anomaly score mitigates the challenge of distinguishing normal from anomalous samples by guiding the CVAE to generate counterfactuals that are coherent and plausible within the learned distribution, thereby indicating which current conditions should change to yield the opposite prediction. For instance, if a counterfactual sample in an anomaly detection model shows significant changes in debt, the user can repay the debt to alter the model's decision. We adopt variational autoencoder driven by its ability to learn complex data distributions and generate realistic samples by mapping data to a latent space and then reconstructing it. This characteristic is particularly beneficial for time series data, where maintaining temporal dependencies and feature relationships is crucial.

To simultaneously address the tasks of anomaly detection and anomaly explanation, we propose **Dual-view experts Routing and Counterfactual generation for explainable time series Anomaly Detection (DRCAD)**, an innovative anomaly detection framework with counterfactual explanations. It leverages dual-perspective learning of data representations and identifies anomalies by comparing the discrepancy between the two representations and a set threshold, effectively avoiding potential biases from relying on reconstruction errors. DRCAD can (1) detect anomalies in input time series data and (2) generate counterfactual samples and assign importance scores to each feature to explain anomalies. The contributions of this paper are summarized as follows:

- We propose DRCAD, a contrastive learning framework where in-patch and patch-wise relationships serve as dual input perspectives for the model. A Flattened Attention mechanism encodes the original data and dual

perspectives, and a Patch Mixture of Experts (MoE) mechanism dynamically adjusts feature importance based on input data. The model detects anomalies by increasing the distance between normal and anomalous points in the representation space.

- We use anomaly scores to constrain the generation of counterfactual samples and combine counterfactual discrepancies with prediction-based contributions to derive feature importance scores, thereby explaining anomalies.
- We evaluate our detection approach with the state-of-the-art anomaly detection algorithms on six real-world datasets, demonstrating better anomaly detection performance. On the SWaT dataset, we compare four representative explanation methods and obtain comparable results. Additionally, we generate importance scores for each feature and provide visual results, showing a high correlation between the most important features and the attack characteristics described in official documents, additional feature-weight experiments corroborate the fidelity of our explanations.

II. RELATED WORK

A. Explainability in Deep Learning

In recent years, the explainability of deep learning networks has garnered significant attention. Traditional machine learning explanation methods, such as decision trees and naive Bayes, have been applied to deep learning models to enhance their explainability. Notable methods like LIME [15], which designs a local surrogate model to fit the model's prediction results, and SHAP [14], which uses Shapley values to calculate each feature's contribution to the model's prediction, are prominent examples. However, these methods face challenges in explaining time series data due to the unique operational patterns that only emerge over observed periods and long-term trends, coupled with the higher information content compared to other data types.

Several methods have been specifically developed to explain time series predictions. TimeX [23] designs a time prediction surrogate model that approximates the original model's output by minimizing the embedding distance in the latent space between the original and surrogate models. FIT [24] estimates feature importance by fixing certain variables and altering others to observe changes in the predicted distribution over time. WinIT [25] extends FIT by using a retrospective window to explain changes in the predicted distribution, illustrating the temporal dependencies between different observations of the same feature for more persistent explanation patterns. Other post-hoc explanation methods generate explanations for model predictions. For instance, counterfactual explanations [16], [17] focus on identifying how features need to change to produce an opposite prediction with minimal perturbation. CounTS [26] designs a self-interpretable time series prediction model that uses causal inference to identify causal relationships between inputs and the predictions, providing feasible counterfactual explanations for time series prediction models. The aforementioned methods are all aimed at explaining the results of time series prediction models. However, merely

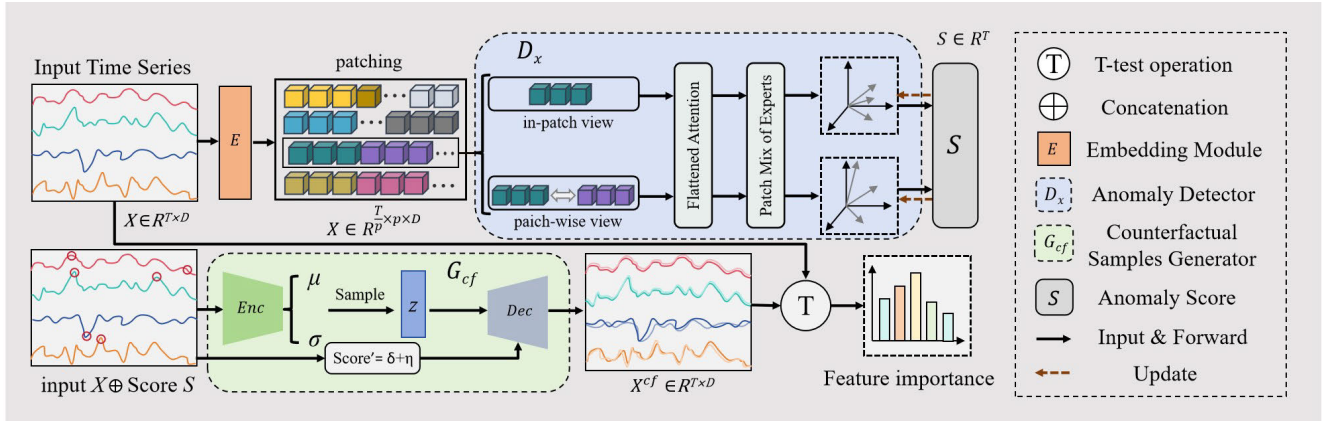


Fig. 1. Overview of DRCAD. E performs basic embedding of the multivariate time series. The detector D_x encodes the in-patch view and the patch-wise view with *Flattened Attention* and fuses them using a *Patch Mixture of Experts*, producing two representations whose discrepancy defines the anomaly score S . The detection decision is obtained by comparing S with a threshold δ . The generator G_{cf} is a score-conditioned CVAE that uses the input and S to generate counterfactual sequences and to compute feature importance for explanation. The workflow is: the embedded series is first partitioned into patches of length p . Next, The detector processes the two views and outputs the score S , which is compared with δ for detection. For explanations, the raw series is concatenated with S to train the score-conditioned CVAE, which then generates counterfactual series under the condition. Feature importance is finally obtained by T-test between the original and the counterfactual series.

providing explanations for the results of time series prediction models is often not directly applicable to time series anomaly detection. This is because the task of anomaly detection requires the ability to accurately identify and interpret anomalous samples, rather than just explaining the overall prediction results.

Currently, only a few works focus on explaining time series anomalies. InterFusion [27] explains anomalies in multivariate time series by using the original reconstruction probability of each detected entity anomaly and applying Markov Chain Monte Carlo imputation to obtain more plausible latent embeddings and reconstructions. However, due to the potential deviation of the reconstruction from the original input data, this may result in explanations that are unrelated to the input data, thereby affecting the accuracy of the interpretation. Some works use attention matrices to explain anomalies. VTT [28] explains anomalies through a comparative analysis of attention patterns derived from the original anomaly-containing data and those obtained from reconstructed data. By analyzing attention maps, key variables and changes in correlations can be identified. However, it lacks fine-grained explanations and fails to provide more specific reasons for anomalies and detailed relationships between variables. Most methods either analyze feature impacts solely based on prediction results or are only suitable for specific models.

B. Time Series Anomaly Detection

Time series anomaly detection aims to identify points or segments in sequence data that deviate from expected patterns. In the statistical domain, techniques include but are not limited to moving averages, exponential smoothing, and autoregressive integrated moving average (ARIMA) models [29]. These traditional methods detect potential anomalies by analyzing historical patterns and trend changes in the data. In the machine learning domain, researchers have explored using clustering algorithms like k-means [30], density-based methods, and classification algorithms such as decision trees [31]

and support vector machines (SVM) for anomaly detection. These methods identify anomalies by learning the similarities and differences between data points.

Deep learning techniques have further expanded the capabilities of anomaly detection due to their powerful feature extraction and high-dimensional data processing abilities [7]. Time series anomaly detection algorithms in deep learning mainly include prediction-based and reconstruction-based methods. In detail, prediction-based algorithms construct a prediction model to estimate future normal data points based on historical data. When a specific data point is observed, the method compares it with the model's predicted value. If there is a significant difference between the observed data point and the predicted value, the point may be considered anomalous. Reconstruction-based algorithms are more widely used in anomaly detection. These methods rely on building a model to encode the normal behavior of a given time series into a low-dimensional latent space and then attempt to reconstruct the original sequence. Since the model is trained only on normal data, it will not effectively reconstruct anomalous subsequences. However, labeling anomalous points in real life is challenging, making it difficult to ensure that the training data is free from anomalies [32]. For prediction-based methods, if the training data contains anomalous samples, the model may incorrectly interpret these anomalous data points as part of normal fluctuations, reducing future prediction accuracy. Reconstruction-based methods typically work by training models to recognize only normal data patterns. If the training set contains anomalous data, the model may learn to reconstruct these anomalous patterns, failing to distinguish between normal and anomalous data. Lately, self-supervised methods like contrastive learning have achieved significant success [33], [34], [35], [36]. Contrastive learning assumes that normal data should exhibit certain similarities, while anomalous data will significantly deviate from the normal data distribution, thereby isolating anomalous samples [6], [10], [13].

TABLE I
NOTATION

Notation	Description
X	The input multivariate time series sequence.
$X_{in-patch}$	In-patch view for detecting anomalies after patching.
$X_{patch-wise}$	Patch-wise view after patching.
$C_S; C_N$	The result after merging the Q and K matrices.
C_{flat}	Reshape the two views to meet the input dimension.
$Flattened_S$	In-patch output results of the flatten operation.
$Flattened_N$	Patch-wise output results of the flatten operation.
$N; S$	Upsampling results of the two views.
$Score$	The anomaly score of the anomaly detection results.
$val_{cf}(\cdot)$	Counterfactual feature importance.
$T(\cdot, \cdot)$	T-test operation.
$val_{pre}(\cdot)$	Feature contribution for detection.
$Norm$	Normalization for calculating feature contributions.
$Stopgrad(\cdot)$	Stop-gradient operation.
η	The parameter used in generating process.

III. METHOD

A. Overview

As illustrated in Figure 1, our goal for multivariate time series is to design a framework that can simultaneously handle anomaly detection and provide explanations for the anomalies. In Table I, we list the notations used in this paper. Overall, we develop an anomaly detection classifier based on contrastive learning, which avoids the issue of large reconstruction losses affecting training, a common problem in reconstruction-based anomaly detection methods. After the model completes its predictions, output the importance of each feature to determine which features are more likely to cause anomalies when altered, we achieve this by calculating an anomaly score for each time window, which serves as a conditional input for the counterfactual generator. This guides the generation of counterfactual samples and adjusts the randomness of counterfactual generation by incorporating the normalized contribution of each feature to the prediction. Sections B and C detail the implementation methods for the anomaly detection model and counterfactual explanations, respectively.

B. Time Series Anomaly Detection

We propose a novel contrastive learning method for time series anomaly detection, leveraging improved representation network techniques without the need for negative examples. The input multivariate time series is defined as $X \in \mathbb{R}^{T \times D}$, with each sample containing time steps T and a number of sensors D . The task of the anomaly detector D_x is to output detection results $y_i^{pred} \in \mathbb{R}^T$. We divide the time series into several patches, splitting the original data into in-patch and patch-wise perspectives. These perspectives model the representations within and between patches, respectively. We use these two perspectives as positive samples for contrastive learning.

To better understand our proposed time series anomaly detection method, we compare it with six popular methods. Figure 2 shows the architecture comparison of six approaches. Rather than relying on conventional multi-head attention and frequency domain analysis mechanisms, DRCAD (Figure 2(f)) uses MLP to “Flatten Attention” in order to enhance feature space resolution and reduce computational complexity. Additionally, during information integration, we introduce an elaborately designed Patch Mixture of Experts (MoE) layer to selectively filter and merge information from the two perspectives.

Here, we introduce the overall structure of the anomaly detection module, including the implementation of the flattened attention mechanism and Patch MoE, and explain how representation differences are used for anomaly detection.

1) *Anomaly Detection Model Structure*: In the Embedding processing stage, we apply instance normalization and channel independence [39] to standardize the multivariate time series input and reduce the model’s parameter count.

Patch was first popularized as a fixed-size, typically non-overlapping image block that is linearly projected into a token [40]. In sequence modeling (NLP), related segmentation ideas appear as chunking units used as tokenization granularity. For time series, a patch is defined as a contiguous, non-overlapping segment along the time axis. It has been shown effective for time series representation learning [39]. To this end, after the embedding operation, the input multivariate time series $X \in \mathbb{R}^{T \times D}$ is partitioned into $\frac{T}{p}$ non-overlapping patch blocks, an operation we denote as $Patching(\cdot)$. Consequently, the original data is transformed from dimensions $T \times D$ to $\frac{T}{p} \times p \times D$:

$$X = Patching(E(X)) \in \mathbb{R}^{\left(\frac{T}{p}\right) \times p \times D}, \quad (1)$$

where $E(\cdot)$ refers to the embedding process. Our approach utilizes a feature projection mechanisms, represented as $FP(\cdot)$, to extract and transform features from two complementary views within our framework. The projection function is realized through a distinct linear transformation:

$$X_{in-patch} = FP(X) \in \mathbb{R}^{d \times \frac{T}{p}}, \quad (2)$$

$$X_{patch-wise} = FP(X) \in \mathbb{R}^{d \times p}. \quad (3)$$

These two views are then used as inputs for the subsequent module, flattened attention, to obtain their respective representations.

The flattened attention module is crucial in our architectural design. This pivotal component facilitates the learning of multi-view representations from input data, enabling a comprehensive analysis of latent patterns. Normal data points tend to exhibit consistent underlying patterns across these varied views. In contrast, anomalies, being rare and lacking definitive patterns, struggle to align with the latent structures of normal data or even with other anomalies. This results in weak correlations between anomalous points and the rest. Consequently, the representational disparities for standard data points across different views remain minimal, whereas anomalies display significant variations. By leveraging crafted criteria to assess

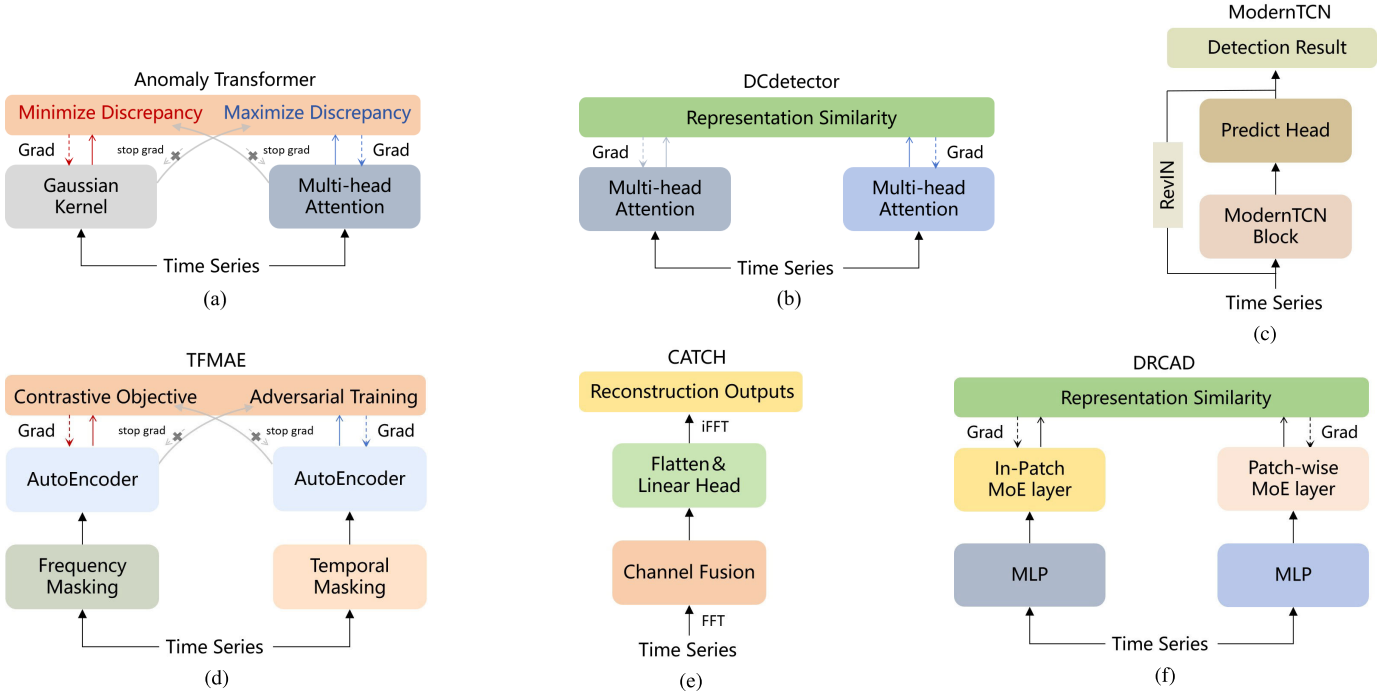


Fig. 2. The distinctions between DRCAD and the other five methods. Anomaly Transformer [6] (a) employs Gaussian kernels to learn prior differences and amplifies these correlation differences using a minimax strategy to quantify each time point. DCdetector [10] (b) employs a “Dual attention” mechanism to differentiate anomalous patterns from regular data points within time series. ModernTCN [37] (c) employs enhanced TCN blocks optimized for time series modeling. TFMAE [13] (d) employs dual Transformer-based autoencoders with temporal and frequency masking strategies to learn unbiased patterns and anomalies. CATCH [38] (e) proposes frequency patchification and a Channel Fusion Module for optimized signal analysis and channel correlation. DRCAD (f) employs a novel approach, utilizing MLP and a specially designed MoE layer to obtain representations of two input views, in contrast to the conventional multi-head attention mechanism.

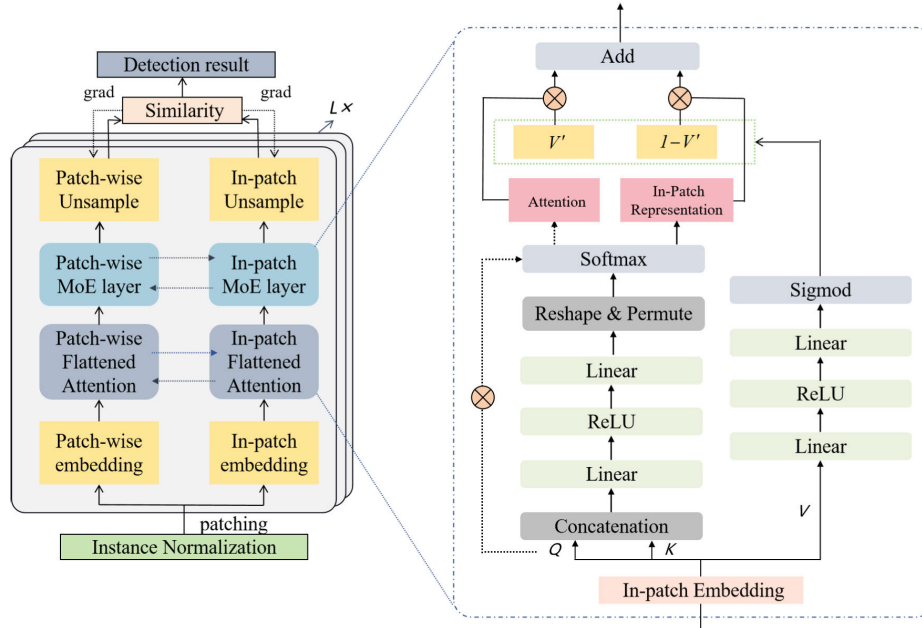


Fig. 3. Structure of the anomaly detector D_x . The detector consists of an Embedding module, a Flattened Attention module, a Patch Mixture of Experts (MoE) module, and an Upsampling operation. The detailed structures of the flattened attention module and the patch MoE module are provided, as they contribute the most to anomaly detection. The structures used for the in-patch and patch-wise views are identical.

these representational differences, we can effectively differentiate between normal and anomalous data points. Finally, we derive anomaly scores by assessing the discrepancies between the two representations and employ a predetermined

threshold for identifying anomalies. Figure 3 shows the specific structure of detector, with the main components being embedding module, flattened attention module, patch MoE module, and the anomaly detection module. Furthermore, the

detailed structures of the flattened attention module and the Patch MoE module are also provided in Figure 3, as they contribute the most to anomaly detection.

2) *Flattened Attention and Patch MoE Structure*: The attention mechanism is effective in capturing patterns in time series data, but its high complexity limits the training efficiency of the model. Unlike previous works that solely rely on the traditional attention mechanism for sequence modeling [6], [10], we employ a MLP to flatten the attention mechanism. Specifically, we replace the traditional dot-product attention with linear layers and non-linear activation functions. Instead of forming all pairwise query-key scores, we flatten tokens and use MLP to compute mixing weights for interactions. This avoids constructing $O(L^2)$ attention maps, reduces memory and compute, and is less sensitive to noisy or irrelevant channels. Together with the Patch Mixture of Experts that adaptively fuses the in-patch and patch-wise views per window, it strengthens the cross-view discrepancy used for scoring and improves both the accuracy and latency trade-off. Specifically, we first initialize and merge the query Q and key K from the two perspectives:

$$Q_S, K_S = W_Q X_{in-patch}, W_K X_{in-patch}, \quad (4)$$

$$Q_N, K_N = W_Q X_{patch-wise}, W_K X_{patch-wise}, \quad (5)$$

$$C_S = \text{Concat}(Q_S, K_S), C_N = \text{Concat}(Q_N, K_N), \quad (5)$$

where $Q_S, K_S \in \mathbb{R}^{\frac{T}{p} \times h \times d_{model}}$, $Q_N, K_N \in \mathbb{R}^{p \times h \times d_{model}}$, and h refer to $\frac{d_{model}}{n_{head}}$.

Next, the concatenated tensor is flattened into two dimensions and passed to a predefined MLP to compute the attention weights. We define the flattening operation C_{flat} as follows, including in-patch view flatten C_{flatS} and patch-wise view C_{flatN} :

$$C_{flatS} = \text{reshape}(C_S) \in \mathbb{R}^{\left(\frac{T}{p}\right) \times h \times (2 \times d_{model})}, \quad (6)$$

$$C_{flatN} = \text{reshape}(C_N) \in \mathbb{R}^{(p) \times h \times (2 \times d_{model})}, \quad (6)$$

where h represent the head number. Our designed MLP consists of two linear layers and a ReLU activation layer. The computation process of the MLP is as follows:

$$M_1 = \text{ReLU}(C_{flat} W_1 + b_1), \quad (7)$$

$$M_2 = M_1 W_2 + b_2. \quad (8)$$

The two-dimensional tensor is then restored to a four-dimensional tensor and transposed, which is:

$$O = (\text{reshape}(M_2))^T. \quad (9)$$

Finally, we obtain the preliminary representation results for the two dimensions, in-patch and patch-wise:

$$\text{Flattened}_S = \text{Softmax}(\text{MLP}(C_{flatS}(C_S))), \quad (10)$$

$$\text{Flattened}_N = \text{Softmax}(\text{MLP}(C_{flatN}(C_N))). \quad (11)$$

Mixture of Experts (MoE) is usually employed in the domain of large language models to enhance the accuracy and capabilities of artificial intelligence systems. We design a Patch MoE layer here to adjust the self-attention weighted results based on the importance of different features in

the input data. Specifically, we generate importance weights related to the input data using the flattened attention module to control the weighted results in the self-attention mechanism, which are defined as follows:

$$\text{FlatAtten}_S = (1 - (\sigma(\text{MLP}(V_S)))) \cdot A_S + \sigma(\text{MLP}(V_S)) \cdot \text{Flattened}_S, \quad (12)$$

$$\text{FlatAtten}_N = (1 - (\sigma(\text{MLP}(V_N)))) \cdot A_N + \sigma(\text{MLP}(V_N)) \cdot \text{Flattened}_N, \quad (13)$$

where V represents the value and A retains the original attention score obtained by multiplying Q and K . This yields the final representation results for the two inputs.

Since the patch-wise view ignores the correlation between points within a patch, and the in-patch view ignores the correlation between different patches, upsampling is required when comparing the results of the two representation networks. For the patch branch, the upsampling operation is performed within the patch (i.e., from patch to point), ultimately obtaining the patch representation. For the in-patch branch, the upsampling operation extends from one patch to the full number of patches, ultimately obtaining the in-patch representation, which is:

$$N = \text{Upsampling}(\text{FlatAtten}_N), \quad (14)$$

$$S = \text{Upsampling}(\text{FlatAtten}_S).$$

3) *Contrastive Difference and Anomaly Detection*: Considering the rarity of anomalies and the common latent patterns shared by normal points, the representations of the same input should be similar. For normal samples, the representations learned from two different perspectives should be highly consistent, but the representations of anomalous samples are difficult to learn, resulting in significant differences in the representations learned from different perspectives. Therefore, in the representation discretization module, we utilize the KL divergence to measure the dissimilarity among the two branches. The loss functions for the two branches are symbolized as follows:

$$L_S = \sum KL(S \parallel \text{Stopgrad}(N)) + KL(\text{Stopgrad}(N) \parallel S), \quad (15)$$

$$L_N = \sum KL(N \parallel \text{Stopgrad}(S)) + KL(\text{Stopgrad}(S) \parallel N), \quad (16)$$

where $\text{Stopgrad}(\cdot)$ represents the stop-gradient operation, S and N represent the embedding results within and between patches, respectively. The final loss function is:

$$L = \frac{L_N - L_S}{\text{len}(N)}. \quad (17)$$

Similar to most anomaly detection methods, we determine whether a point is an anomaly by comparing the anomaly score with a threshold. The anomaly score is the degree of difference between the two perspectives:

$$\text{Score}(X_i) = \sum KL(S \parallel \text{Stopgrad}(N)) + KL(N \parallel \text{Stopgrad}(S)), \quad (18)$$

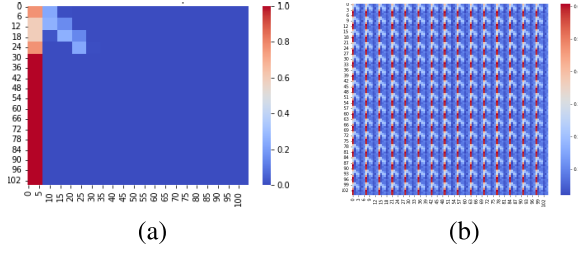


Fig. 4. Visualization results of different detection model on 105 window size. (a) Higher attention weights are assigned to temporally proximate time series data points, while correlations between features further apart in the sequence are disregarded. (b) The attention weights are overly smoothed, resulting in the neglect of crucial areas within the time series. Consequently, neither yields the desired level of explanatory granularity.

where $KL(\cdot \| \cdot)$ is the KL divergence. The anomaly score is compared with a prior threshold δ , and points exceeding the threshold are considered anomalies. That is:

$$y_i = \begin{cases} 1: \text{anomaly} & \text{Score}(X_i) \geq \delta \\ 0: \text{normal} & \text{Score}(X_i) < \delta. \end{cases} \quad (19)$$

C. Counterfactual Feature Importance Explanation

In this section, we introduce the method for explaining anomalies by comparing the differences between counterfactual and original samples to obtain feature importance scores. A higher score indicates a greater contribution of that feature to the occurrence of the anomaly. Notably, although our anomaly detection model incorporates an attention mechanism, we chose not to use the attention matrix for explanations. We observe that, under a sliding-window setting, the attention weights mainly concentrate on outputs from the adjacent window, yielding only local explanatory effects, as Figure 4 shows. In contrast, counterfactual explanations aim to provide insights into what changes to the input might lead to a desired prediction. Such altered inputs shift the prediction to a specific target, for example, from normal (prediction value 0) to anomalous (prediction value 1), thus offering actionable explanations for the anomaly.

Considering that existing counterfactual generators mostly satisfy the requirement of proximity without providing answers on which features to change, we acknowledge that constraining features is challenging. In the real world, it is challenging to pre-determine the importance and range of feature changes. This makes generating feature importance scores by simply comparing the differences between generated samples and original samples more random. Therefore, we enhance the model by predicting feature importance, making the explanation results more meaningful by overlaying the normalized feature mask on the prediction results.

We introduce the process of generating counterfactual samples in paragraph a). In paragraph b), we describe the process of calculating feature importance after obtaining the generated samples. In paragraph c), we present the final method used to explain anomalies.

1) *Generating Counterfactual Samples:* We use a Conditional Variational Autoencoder (Conditional VAE) [41] to generate counterfactual samples. For input multidimensional

time series, the trained anomaly detection model scores the sample points as $Score \in \mathbb{R}^{T \times D}$, and outputs the detection results $y_i^{pred} \in \mathbb{R}^T$. The objective is to generate a set of samples $x_i^{cf} \in \mathbb{R}^{D \times T}$ that make minimal changes to the features but can flip the model's prediction results, for example, changing $y_i^{pred} = 0$ to $y_i^{cf} = 1$.

Unlike previous methods for generating counterfactual samples, we use the anomaly score S as a condition to supervise the learning process of the autoencoder. We first embed the input score, encoding it as $Score' \in \mathbb{R}^{T \times 2}$. In the encoder part, we input $Score'$ as a conditional parameter along with the time series sample to obtain the mean μ and variance σ . Similarly, in the decoder part, the condition is also input along with μ and σ to generate the counterfactual sample. During training, the CVAE attempts to generate samples from the latent space distribution that satisfy the conditional constraints. To meet the proximity requirement, we add an L1 norm as a regularization constraint. Therefore, the final loss $\mathcal{L}$ is:

$$\mathbb{E}_{q_\phi(z|x_i, Score')} [-\log p_\theta(x_i|z, Score')] + \lambda_1 \|X^{cf} - X\|_1 + KL(q_\phi(z|x_i, Score') \| p(z|Score')), \quad (20)$$

where $Score'$ is the conditional input to the variational autoencoder, and X^{cf} is the generated counterfactual sample. After training the model, we can generate counterfactual samples needed for the desired prediction of anomalies by elevating the given condition, that is:

$$Score' = \delta + \eta, \quad (21)$$

where δ is prior threshold and η is the bias of elevating.

2) *Feature Discrepancy:* After obtaining the counterfactual samples, we can compare the data changes between the original data and the generated counterfactual samples to obtain the feature discrepancies. To ensure the entire process is interpretable, we use an independent sample T-test to compare each feature of the original and generated samples, calculating their significant differences. The T-test is a statistical method used to compare whether the means of two independent samples are significantly different. By calculating the T-statistic and the corresponding P-value, the statistical relevance of feature discrepancies between the paired samples can be ascertained.

For the independent sample T-test, we establish two competing hypotheses: Null hypothesis (H0): Both sample groups share identical mean values; Alternative hypothesis (H1): The mean values of the two samples differ. The decision to accept or reject H0 hinges on the calculated T-statistic and its corresponding P-value. When the P-value falls below the pre-determined significance threshold (typically 0.05), we dismiss H0, suggesting a noteworthy distinction between the sample means. The magnitude of this difference is inversely related to the P-value, smaller P-values indicate more pronounced feature disparities.

Our goal is to obtain the importance score for each feature based on the discrepancy between the two samples. Specifically, for each feature, we extract the values of that feature from the original and generated samples, forming two independent samples with a sample size of n_T . First, we calculate the pooled standard deviation, then compute the

Algorithm 1 Explaining DRCAD's Detection Result

Input: Multivariate time series $X \in \mathbb{R}^{T \times D}$, detection results $Score$, the CVAE model, elevating bias η .

Output: Feature importance values $val_{cf} \in \mathbb{R}^{1 \times D}$, the well-trained CVAE, encoding network parameters θ , decoding network parameters ϕ , counterfactual examples $X^{cf} \in \mathbb{R}^{T \times D}$.

```

1: # Incorporate anomaly Score condition into the model.
2:  $Score' \leftarrow \text{ReLU}(\text{Linear}(Score))$ .
3: Concatenate  $X$  with  $Score'$  in Encoder.
4: Concatenate  $z$  with  $Score'$  in Decoder.
5: # Initialization parameters.
6: CVAE,  $\theta$ ,  $\phi \leftarrow$  Randomly initialize the parameters.
7: # Training Process of CVAE.
8: for  $i = 1, 2, \dots, e_p$  do
9:    $z \leftarrow \text{Sample}(\mu, \log \sigma)$ .
10:  Calculate  $\mathcal{L}$  via Eq.(20).
11:  CVAE,  $\theta$ ,  $\phi \leftarrow$  Update parameters using Adam.
12: end for
13: Generating  $X^{cf}$  by elevating  $Score'$  to  $\delta + \eta$ .
14: # Calculate the feature importance.
15: for  $i = 1, 2, \dots, D$  do
16:  Calculate the feature discrepancy by Eq.(23).
17:  Calculate the contribution of each feature by Eq.(24).
18:  Output  $i$ -th feature importance by Eq.(25).
19: end for

```

T-statistic for the two samples. The formula for calculating the T-statistic is:

$$T(X_1, X_2) = \sqrt{\frac{n_T}{2}} \frac{E(X_1) - E(X_2)}{S}, \quad (22)$$

and we define the counterfactual discrepancy for each feature as:

$$val_{cf}(i) = T(x_i, x_i^{cf}). \quad (23)$$

Using the T-statistic distribution table, we can find the corresponding P-value. The smaller the P-value, the more significant the discrepancy in that feature between the generated datasets and the origin. This allows us to evaluate the importance of each feature by comparing the differences between the original and generated samples.

3) *Explanation by Feature Importance:* Algorithm 1 illustrates the computational process of feature importance. The counterfactual generator captures the relationships between time series features, but it does not impose constraints when changing feature values. This can lead to an excessive number of undesirable explanations (e.g., the model may prefer to change only a few features, or some features may lose discreteness during the change process). Our solution is to add an additional feature importance prediction. We calculate the impact of each feature on the prediction result. Specifically, after the model training is completed, we sequentially transform each feature into random noise, erasing its effect on the prediction. After obtaining the true prediction value and the actual prediction value, we normalize the impact of each feature on the prediction. The formula about feature contribution is defined as follows:

$$val_{pre}(i) = \text{Norm}(D(X_{I/(i)})), \quad (24)$$

where I represents the collection of all feature sequences. Finally, we perform a weighted summation to determine the final feature importance value:

$$val(i) = val_{pre}(i) - \lambda_2 \cdot val_{cf}(i), \quad (25)$$

where λ_2 aligns the numerical values. Feature importance highlights the suspicious features causing anomalies and provides feasible actions on how altering features could lead to anomalies.

IV. EXPERIMENTS

A. Datasets

We adopt six widely used datasets to evaluate the anomaly detection model and provide explanation results on the SWaT dataset. Details of datasets used are summarized in TABLE II, including:

- **PSM [42]:** This dataset is collected internally from multiple application server nodes at eBay and contains 25 dimensions.
- **MSL [43]:** NASA's collection encompasses data on the Mars rover's sensor health and actuator operations, offering valuable insights into the vehicle's performance.
- **SMAP [43]:** This dataset, also gathered by NASA, includes soil samples and telemetry data utilized by the Mars rover.
- **SWaT [44]:** This dataset is obtained from 51 sensors of the critical infrastructure system under continuous operations.
- **SMD [45]:** This dataset is a five-week-long collected from an internet company compute cluster, which stacks accessed traces of resource utilization of 28 machines.
- **GECCO [46]:** This dataset focuses on drinking water quality in the context of the Internet of Things.

B. Baselines

We compare our model with seventeen compelling anomaly detection models, including:

- **Reconstruction-based models:** LSTM-VAE [47], OmniAnomaly [45], BeatGAN [48], InterFusion [27], CATCH [38].
- **Autoregression-based models:** VAR [49], LSTM [43], CL-MPPCA [50], GPT4TS [51], ModernTCN [37].
- **Density-estimation models:** LOF [52], MPPCACD [53], DAGMM [54].
- **Clustering-based methods:** DeepSVDD [55], THOC [56], ITAD [57].
- **Classic methods:** OCSVM [50], IForest [58].
- **Change point detection and time series segmentation methods:** BOCPD [59], U-Time [60], TS-CP2 [61].
- **Contrastive learning based methods:** Anomaly Transformer [6], DCdetector [10], TFMAE [13].

We compare our approach with four prevalent counterfactual generation methods on SWaT dataset, that is:

- **Regularized Gradient Descent (RGD):** [17] A approach that minimize the sum of squared differences between expected outcomes and counterfactuals, employing gradient descent for counterfactual search.

TABLE II
SPECIFIC INFORMATION ABOUT THE DATASETS. AR (ANOMALY RATIO) IS THE PROPORTION OF ANOMALIES. THE FIRST FOUR DATASETS HAVE RELATIVELY HIGHER AR, WHEREAS SMD AND GECCO HAVE LOWER AR

Benchmark	Source	Dimension	Window	#Training	#Test(Labeled)	AR(%)
MSL	NASA Space Sensors	55	90	58,317	73,729	10.5
SMAP	NASA Space Sensors	25	105	135,183	427,617	12.8
PSM	eBay Server Machine	25	60	132,481	87,841	27.8
SWaT	Infrastructure System	51	105	495,000	449,919	12.1
SMD	Internet Server Machine	38	105	708,405	708,420	4.2
GECCO	Water Quality for IoT	9	90	69,260	69,261	1.1

- **Generative Adversarial Network (GAN):** [62] This approach integrates the standard GAN with a target classifier.
- **Counterfactual Search Guided by Prototypes (CSGP):** [63] This method extends RGD by pushing counterfactuals towards more realistic data points of the desired class using class prototypes.
- **Generating Counterfactuals using Residual GANs (CounterGAN):** [16] The CounterGAN method leverages a residual generative adversarial network to generate counterfactuals for a target classifier.

C. Evaluation Metrics

1) *Metrics on Time Series Anomaly Detection:* We employ accuracy, precision, recall, F1-score which are the most commonly used evaluation metrics for anomaly detection tasks.

2) *Metrics on Counterfactual Examples Generation:* We utilize four widely adopted metrics from related work to evaluate the relative performance of our method for generating counterfactual time series samples. These metrics include realism, sparsity, prediction gain (pred-gain), and latency.

Realism reflects the extent to which the content or results generated by counterfactuals are consistent or believable. We employ the L2 norm to calculate reconstruction error as a measure of realism. A lower value indicates higher realism of the method.

Sparsity refers to the degree of difference between counterfactuals and the actual observed data. Lower sparsity implies that counterfactuals are closer to actual data points. We use the L1 norm to quantify this difference.

Prediction gain shows the contrast between the classifier's predictions on the counterfactual examples and the original input data point. A higher value indicates a higher proportion of normal samples successfully reversed into anomalies.

Latency represents the computational cost of generating counterfactual samples. In many scenarios of time series anomaly detection, generating explanatory samples is required to be as real-time as possible.

D. Implementation Details

We document the hyperparameters used, as well as the run-time environment and hardware configuration. In the anomaly detection module, the default epoch is set to 1, and the learning

rate is set to $1e-4$. The CVAE model training data is stored as time series sampled from anomalous samples according to the time window, with duplicates present. We set epoch = 5, the training-to-validation ratio to 8:2, $\lambda_1 = 0.1$, $\lambda_2 = 1e-3$, learning rate to $1e-4$, η is set to $5e-4$ and batch size is set to 128. The encoder and decoder each have 3 layers. During testing, we use normal samples. Both the anomaly detection module and the CVAE module are implemented in PyTorch [64] and optimized using the Adam [65] optimizer. The hardware used is an NVIDIA GeForce RTX 3090 24G.

E. Main Results

1) *Anomaly Detection Results:* We compare our model with 17 representative methods on four higher anomaly ratio (AR) datasets using precision (P), recall (R), and F1-score (F1) as the evaluation metrics. Table III reports the results. The evaluation results of CATCH on the SMAP dataset are omitted from this study due to the lack of preprocessed data and a standardized data processing methodology. The comparison shows that our detector achieves state-of-the-art (SOTA) performance on multiple benchmarks. In addition, on the two lower AR datasets GECCO and SMD, our method maintains high precision and competitive F1, as shown in Table IV.

Although the DRCAD model performs exceptionally across multiple evaluation metrics, its recall rate is slightly lower than other methods. One possible explanation is that the designed Patch MoE is overly stringent, causing some borderline anomalous data to be filtered out and thus not recognized as anomalies by the model. While this stringent filtering standard enhances the overall reliability of the model, it may also impact the recall rate to some extent.

2) *Counterfactual Generation:* In this subsection, we evaluate the generation capability of the CVAE and the quality of counterfactual samples on the SWaT dataset. The CVAE is trained using anomalous data to avoid the absence of high-scoring anomalies during training, allowing the model to learn the input condition score.

We compare our interpretation method with four popular methods in terms of proximity, sparsity [66], prediction gain, and time efficiency. TABLE V shows our comparison results on SWaT dataset. DRCAD's prediction gain is slightly lower than RGD because the VAE model has higher randomness when generating counterfactual samples.

TABLE III

COMPREHENSIVE DETECTION OUTCOMES ON FOUR REAL-WORLD TIME SERIES DATASETS WHICH HAVE RELATIVELY HIGHER AR, RANKED IN ASCENDING ORDER OF PERFORMANCE. THE METRICS P, R, AND F1 DENOTE PRECISION, RECALL, AND F1-SCORE, RESPECTIVELY. ALL VALUES ARE PRESENTED AS PERCENTAGES, WITH THE TOP RESULTS IN BOLD AND THE SECOND-BEST UNDERLINED

Dataset		MSL			SMAP			SWaT			PSM		
Metric	Venue	P	R	F1	P	R	F1	P	R	F1	P	R	F1
LOF [52]	MOD-00	47.72	85.25	61.18	58.93	56.33	57.60	72.15	65.43	68.62	57.89	90.49	70.61
IForest [58]	ICDM-08	53.94	86.54	66.45	52.39	59.07	55.53	49.29	44.95	47.02	76.09	92.45	83.48
Deep-SVDD [55]	ICML-18	91.92	76.63	83.58	89.93	56.02	69.04	80.42	84.45	82.39	95.41	86.49	90.73
DAGMM [54]	ICLR-18	89.60	63.93	74.62	86.45	56.73	68.51	89.92	57.84	70.40	93.49	70.03	80.08
U-Time [60]	NeurIPS-19	57.20	71.66	63.62	49.71	56.18	52.75	46.20	87.94	60.58	82.85	79.34	81.06
CL-MPPCA [50]	KDD-19	73.71	88.54	80.44	86.13	63.16	72.88	76.78	81.50	79.07	56.02	99.93	71.80
BeatGAN [48]	IJCAI-19	89.75	85.42	87.53	92.38	55.85	69.61	64.01	87.46	73.92	90.30	93.84	92.04
OmniAnomaly [45]	KDD-19	89.02	86.37	87.67	92.49	81.99	86.92	81.42	84.30	82.83	88.39	74.46	80.83
ITAD [57]	CIKM-20	69.44	84.09	76.07	82.42	66.89	73.85	63.13	52.08	57.08	72.80	64.02	68.13
THOC [56]	NeurIPS-20	88.45	90.97	89.69	92.06	89.34	90.68	83.94	86.36	85.13	88.14	90.99	89.54
InterFusion [27]	KDD-21	81.28	92.70	86.62	89.77	88.52	89.14	80.59	85.58	83.01	83.61	83.45	83.52
AnomalyTrans [6]	ICLR-22	91.92	96.03	93.93	93.59	99.41	96.41	89.10	99.28	94.22	96.94	97.81	97.37
DCdetector [10]	KDD-23	92.21	<u>97.08</u>	<u>94.58</u>	94.47	<u>98.77</u>	96.57	93.16	99.96	96.44	97.19	97.53	97.36
GPT4TS [51]	NeurIPS-23	81.72	81.10	81.41	90.07	55.77	68.89	92.10	93.08	92.59	97.39	94.13	95.73
ModernTCN [37]	ICLR-24	89.66	75.19	81.79	90.23	55.94	69.06	95.64	90.02	92.84	98.60	94.39	96.45
TFMAE [13]	ICDE-24	<u>92.41</u>	95.00	93.69	<u>95.17</u>	98.58	<u>96.84</u>	<u>95.16</u>	97.89	<u>96.51</u>	96.99	97.83	<u>97.41</u>
CATCH [38]	ICLR-25	64.77	94.98	77.02	-	-	-	91.65	93.44	92.57	96.13	95.51	95.82
Ours	-	92.49	98.82	95.55	96.10	98.32	97.20	93.95	<u>99.77</u>	96.77	<u>97.43</u>	<u>98.78</u>	98.10

TABLE IV

DETECTION RESULTS ON GECCO AND SMD DATASETS. THESE TWO DATASETS HAVE A LOWER AR THAN THE OTHER FOUR DATASETS

Dataset	GECCO			SMD		
	P	R	F1	P	R	F1
OCSVM [50]	18.5	74.3	29.6	44.3	76.7	56.2
IForest [58]	43.9	35.3	39.1	42.3	73.3	53.6
AnomalyTrans [6]	25.7	28.5	27.0	<u>89.9</u>	90.2	90.0
DCdetector [10]	37.6	55.8	<u>45.0</u>	84.3	81.3	82.8
GPT4TS [51]	20.3	54.8	29.6	89.6	81.1	85.2
ModernTCN [37]	41.9	54.7	47.5	76.2	79.9	78.0
TFMAE [13]	43.8	54.3	<u>48.5</u>	90.1	89.1	<u>89.6</u>
CATCH [38]	<u>44.0</u>	27.4	33.8	81.8	<u>90.2</u>	85.8
Ours	44.2	<u>57.0</u>	49.8	88.4	86.7	87.6

3) *Feature Importance and Explanation*: We aim to explain which features are more likely to cause decision reversals by outputting feature importance. Therefore, we use T-test to measure importance, as Figure 5 shows. The lower the p-value, the more important the feature. Then obtain the normalized results of each feature's impact on the final prediction on the SWaT dataset. According to the formula 25, we calculate the final feature importance values, as shown in Figure 6. Note that the proposed method is model-agnostic and data-agnostic. We

TABLE V

COUNTERFACTUAL TIME SERIES EVALUATION RESULTS ON THE SWaT DATASET. THE ARROWS DENOTE WHETHER LARGER ↑ OR LOWER ↓ VALUES ARE BETTER

Method	SWaT			
	Realism↓	Sparsity↓	Pred-gain↑	Latency(ms)↓
RGD [17]	2.954	3.505	0.789	622.843
GAN [62]	3.306	32.577	0.420	0.733
CSGP [63]	<u>2.424</u>	<u>3.080</u>	0.478	1442.730
CounterGAN [16]	3.678	13.920	0.620	0.009
DRCAD	0.434	0.862	<u>0.710</u>	<u>0.697</u>

chose to provide explanations on the SWaT dataset because it has more authoritative official documentation.

We select the top four features with the highest scores from the obtained feature importance results to illustrate the rationality of the explanation: P302, MV303, DPIT301, and AIT504. Higher feature importance scores indicate that modifying these features are more likely to cause anomalous events. The feature with the highest importance score is P302, indicating that if we modify the value of P302 beyond a certain range, the model is highly likely to predict the time series result as anomalous. According to the description provided by the SWaT dataset [44], P302 was deliberately closed from 31/12/2015 01:45:19 to 11:15:27. This actuator pumps water

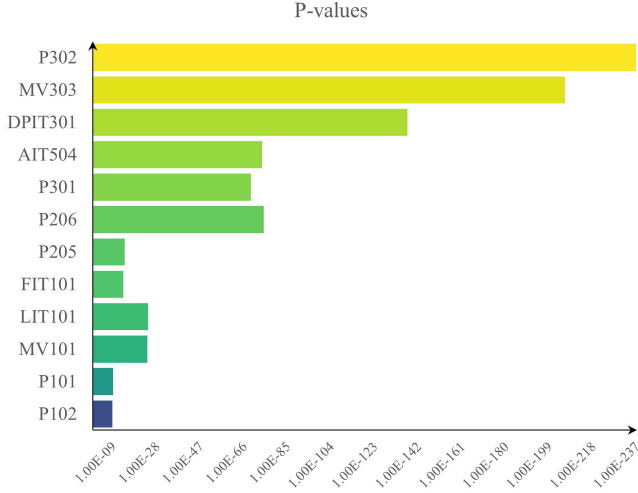


Fig. 5. Feature p-values ranking. A lower p-value indicates that the feature undergoes larger changes when generating counterfactuals.

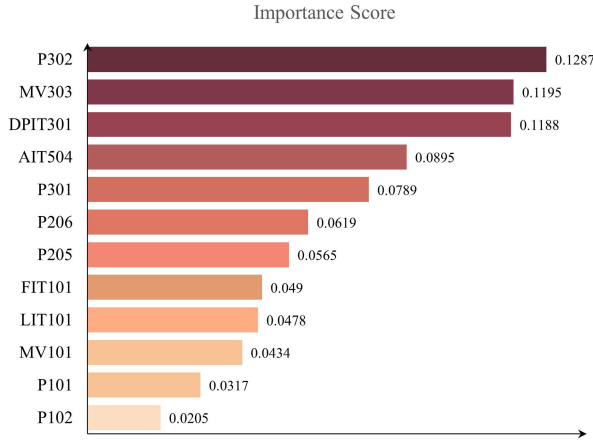


Fig. 6. Feature importance ranking on the SWaT dataset.

from the UF feed tank to the RO feed tank through UF filtration. The negative impact of attacking actuator P302 on the water flow of the RO unit is that tank T401 stops inflowing, affecting the feature values after P4, as shown in Figure 9. From 29/12/2015 14:38:12 to 14:50:08, MV303 is prevented from opening, causing changes in the backwash process and leading to the cessation of stage 3, resulting in anomalies.

By attributing features to decisions, we effectively avoid the randomness in the counterfactual generation process. Relatively, MV303 have a high proportion in the normalized feature impact on prediction results, which is one of the key reasons we can identify it as a cause of anomalies. Similarly, DPIT301 causes the water level of T401 to drop and the water level of T301 to rise by setting the value > 40 kpa. AIT504 is modified twice within twelve minutes, causing water to not enter the drainage pipe, which fully aligns with the attack methods described in the documentation. This indicates that our results are highly credible.

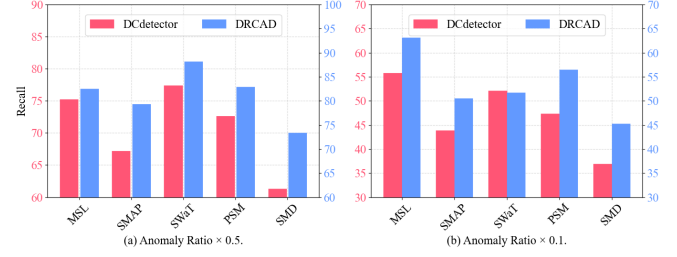


Fig. 7. Recall comparison under different anomaly ratios. (a) $AR \times 0.5$: anomaly prevalence reduced to 50% of the original; (b) $AR \times 0.1$: reduced to 10%. Results for 5 datasets comparing DCdetector and DRCAD. As AR decreases, recall drops for both methods, with a clearer advantage for DRCAD at $AR \times 0.1$.

F. Model Analysis

1) *Anomaly Ratio Sensitivity*: Anomaly ratio (AR) is considered a key factor that controls how hard detection becomes. When anomalies are fewer, recall usually drops and it is harder to balance recall and precision. To study this effect, we create two test settings named $AR \times 0.5$ and $AR \times 0.1$ while keeping the raw signals unchanged. For each setting we choose the decision threshold on a validation set using the precision recall curve so that the operating point is fair for every method. We evaluate on MSL, SMAP, SWaT, PSM, and SMD and report per dataset results with recall trends shown in Figure 7.

The results show a clear two step drop as the anomaly ratio goes from the original level to $AR \times 0.5$ and then to $AR \times 0.1$. Both DCdetector and DRCAD lose recall, yet DRCAD stays higher on all datasets and the gap is larger at $AR \times 0.1$. The advantage is strongest on harder datasets such as SMD where rare events are easy to miss. This result may be due to the design of DRCAD, where flattened token mixing reduces sensitivity to channel noise and the Patch MoE adaptively fuses in-patch and patch-wise views to retain weak but useful cues.

2) *Ablation Studies on Detector Structure*: Table VI shows the ablation studies on the Flattened Attention and Patch MoE modules. “No Flatten” indicates that the representations of the two views are obtained through the traditional multi-head attention mechanism, while “No MoE” means replacing with multi-scale patch sizes, specifically patch size is set to 3 and 5. Experimental results indicate that both designed modules can enhance performance. Patch MoE is more prominent because it integrates useful information during representation extraction.

3) *Ablation Studies on Dual-View Structure Design*: We conduct an investigation into the rationality of homogeneous branch structures, as shown in Table VII, while “Original” refers to no modification; “Freeze” denotes fixing the parameters of the branch; “Random init” represents random initialization of the branch; “Switch” indicates interchanging inputs between two branches. To facilitate a more comprehensive analysis of feature disparities across varying viewpoints between the dual branches, we introduced C-S and R-A-R metrics [67], where C-S stands for Cosine Similarity, while R-A-R represents Range-AUC-ROC based

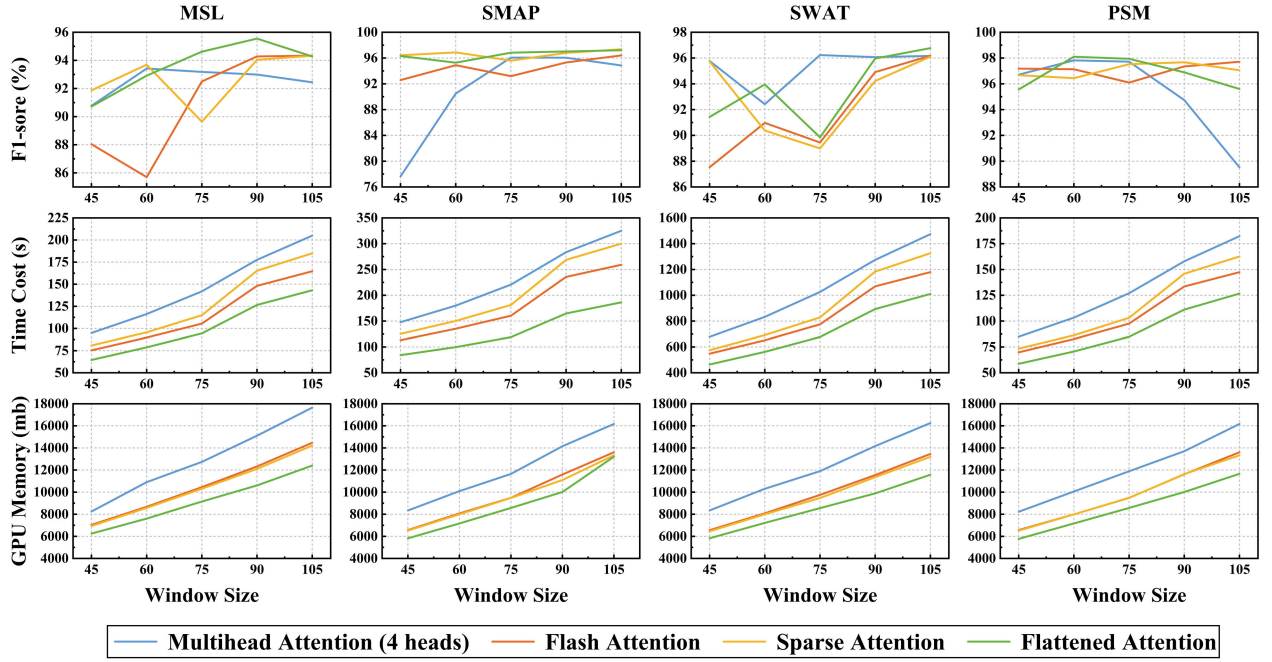


Fig. 8. Comparative results of employing four attention mechanisms in the detector across four datasets, respectively.

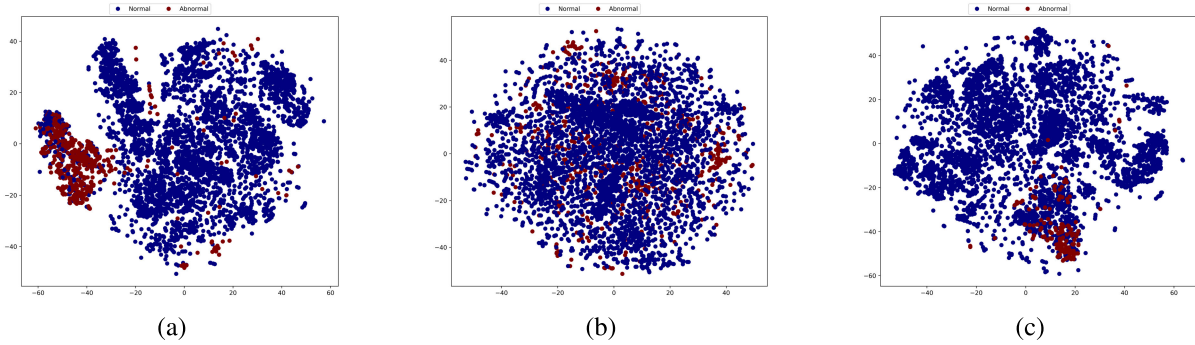


Fig. 9. Visualization results of latent distribution under distinct feature weights by T-SNE.

TABLE VI
ABLATION STUDIES ON FLATTENED ATTENTION AND PATCH MoE MODULE

Module		MSL			SMAP			SWaT			PSM		
Flatten	MoE	P	R	F1	P	R	F1	P	R	F1	P	R	F1
YES	No	91.73	91.99	91.86	92.19	96.00	94.06	92.15	96.49	94.27	97.25	86.60	91.62
NO	YES	94.60	96.03	95.26	96.89	98.48	96.47	92.81	99.43	96.01	97.79	97.39	97.59
NO	NO	92.21	97.08	94.58	94.47	98.77	96.57	93.16	99.96	96.44	97.19	97.53	97.36
YES	YES	92.49	98.82	95.55	96.10	98.32	97.20	93.95	99.77	96.77	97.43	98.78	98.10

on label transformation under ROC curve. It can be seen that freezing either the in-patch or patch-wise branch leads to a slight decrease in model performance and similarity metrics, with values approaching those observed in the unfrozen state. This phenomenon can demonstrate that both branches capture equally important yet distinct features. The observed increase in the R-A-R metric may be due to the

model's enhanced focus on optimizing the unfrozen branch when one branch remains static. Similarly, the R-A-R metric potentially benefits from this stability, as it considers performance across various thresholds, resulting in a marginal performance improvement. When initialization operations are performed, a significant decrease in metrics is observed. This substantial reduction can be explained by the disruption

TABLE VII
ABLATION STUDIES ON HOMOGENEOUS BRANCH STRUCTURES

Module		MSL			SMAP			SWaT			PSM		
In-patch	Patch-wise	F1	C-S	R-A-R	F1	C-S	R-A-R	F1	C-S	R-A-R	F1	C-S	R-A-R
Original	Original	95.55	62.35	89.15	97.20	42.63	95.59	96.77	67.42	94.83	98.10	24.95	91.26
Freeze	Original	92.42	62.03	91.82	95.83	42.46	95.83	93.87	66.46	93.01	95.04	22.22	88.73
Original	Freeze	91.00	62.33	88.39	96.02	42.30	93.39	90.77	67.49	96.72	97.18	22.33	87.76
Random init	Original	67.89	51.75	63.30	92.40	36.20	85.08	68.35	58.91	83.45	79.67	28.44	79.31
Original	Random init	66.95	53.00	62.41	92.13	37.82	83.16	75.72	54.63	85.35	78.95	29.19	82.13
	Switch	92.41	62.78	86.06	96.85	41.95	96.11	94.61	67.14	92.95	97.58	24.92	90.34

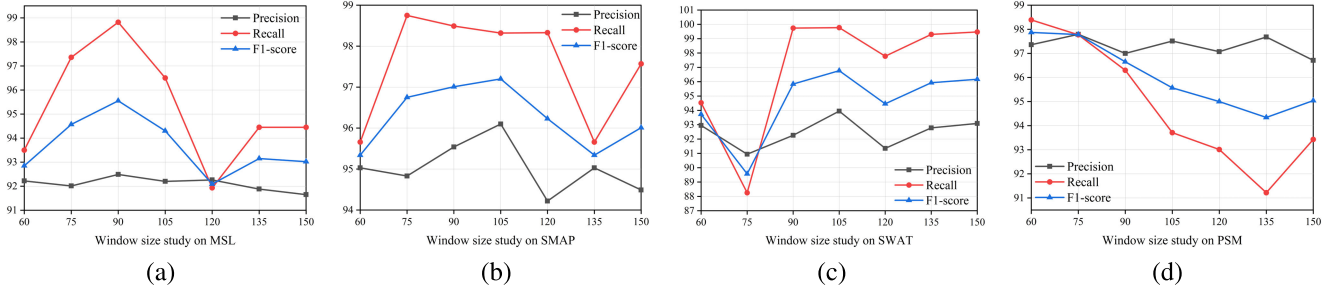


Fig. 10. Visualization results of the different window size comparison.

of pre-trained weights and the consequent amplification of disparities between the two branches. Specifically, there are no notable changes when switching the inputs from the two views, confirming that identical branch structures are capable of capturing features from different views, as discussed above.

4) *Ablation Studies on Attention Mechanism:* Moreover, we perform ablation studies on Attention Mechanism to evaluate the efficiency and necessity of our proposed design as the Flattened Attention is developed to optimize parameter efficiency while retaining the capability to extract temporal patterns from sequential data. Figure 8 illustrates the results of modeling time series using three different attention mechanisms in the detector: Multi-head Attention [68], Flash Attention [69], and Sparse Attention [70]. It can be seen that Flattened Attention exhibits superior performance enhancement, more efficient runtime, and smaller memory consumption. Flash Attention and Sparse Attention have the same level of complexity; however, Sparse Attention demonstrates superior stability. Furthermore, it is noteworthy that the multi-head attention mechanism did not demonstrate superior performance in comparative analyses. This underperformance may be attributed to overfitting, as the datasets like SMAP and PSM provided relatively limited training data. This phenomenon underscores the critical balance between model complexity and data availability required for optimal performance.

5) *Feature Importance Analysis:* We validate the efficacy of the feature importance output. Figure 9 illustrates the visualization of feature distributions in the SWaT dataset under three conditions. Figure 8(a) visualizes the distribution of

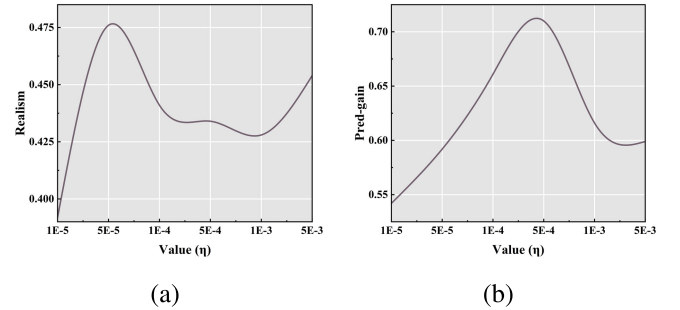


Fig. 11. Visualization results of different elevating bias η for counterfactual generation.

normal and anomalous points under baseline conditions, where red points denotes anomalous and blue represents normal instances. Figure 8(b) demonstrates the result after artificially reducing the weights of the top five features ranked by importance, revealing that anomalous points become significantly less distinguishable. Figure 8(c) shows the outcome of randomly downweighting five features across all dimensions, where the distribution overlap between normal and anomalous points in the low-dimensional embedding space exhibits minimal variation. This comparative analysis validates the effectiveness of feature importance ranking methodologies in maintaining class separability for time series anomaly detection tasks.

6) *Parameter Selection:* We investigate the parameter sensitivity of the DRCAD model. Figure 10 demonstrates detect performance across different window sizes, indicating robust stability of our model across four datasets. Within the window

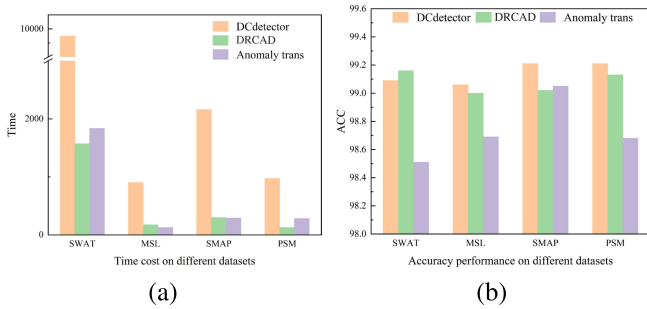


Fig. 12. (a) Time cost comparison on different datasets. (b) Accuracy performance on different datasets.

size range [60,105], fluctuations in DRCAD performance do not exceed 8.03%. Figure 11 illustrates the performance of counterfactual generation under varying elevating bias η values. Lower Realism and Pred-gain metrics indicate reduced perturbation in the generated counterfactual samples. Note that excessively high bias values may compromise the integrity of the counterfactual generation rules.

7) *Time Cost and Accuracy (ACC) Comparison:* In Figure 12 we compare the Time efficiency and Accuracy (ACC) of our method with two highly correlated methods, Anomaly Transformer and DCdetector. Across four datasets, our method yield the lowest average time overhead, and ACC results close to DCdetector, higher with Anomaly Transformer. This can be attributed to our designed flattened attention mechanism, which successfully ties the algorithm's complexity to the hidden layer dimensions of the MLP, thereby avoiding excessive computations.

V. CONCLUSION AND FUTURE WORK

In this paper, we propose DRCAD, a novel approach that addresses both the challenges of multivariate time series anomaly detection with the need for result interpretation. We present an innovative high-confidence explanation technique applicable to time series anomaly detection model. Specifically, our method employs a contrastive representation learning framework to capture time series dependencies and identify anomalies by analyzing representational discrepancies from multiple angles. Furthermore, we develop counterfactual instances based on detected anomalies and quantify feature importance through the extent of feature alterations. Experimental evaluations demonstrate that DRCAD achieves state-of-the-art outcomes across various metrics in anomaly detection tasks and superior realism and sparsity in counterfactual generation compared to existing methods, as validated on four real-world datasets, the explanations on the SWaT dataset align closely with the official documentation.

However, there are still many aspects in our work that require improvement. One observed limitation is that our explanation process does not account for the differences between different types of features. In practice, as seen with the SWaT dataset, discrete features should be given greater weight because they influence the data of subsequent features.

REFERENCES

- [1] T. Fernando, H. Gammulle, S. Denman, S. Sridharan, and C. Fookes, "Deep learning for medical anomaly detection—A survey," *ACM Comput. Surv.*, vol. 54, no. 7, pp. 1–37, Sep. 2022.
- [2] A. A. Cook, G. Misirli, and Z. Fan, "Anomaly detection for IoT time-series data: A survey," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6481–6494, Jul. 2020.
- [3] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "RFG-HELAD: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5895–5910, 2024.
- [4] P.-F. Marteau, "Random partitioning forest for point-wise and collective anomaly detection—Application to network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2157–2172, 2021.
- [5] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [6] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–20.
- [7] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," *Proc. VLDB Endowment*, vol. 15, no. 9, pp. 1779–1797, May 2022.
- [8] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," 2019, *arXiv:1901.03407*.
- [9] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," 2022, *arXiv:2211.05244*.
- [10] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "DCdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. 29th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2023, pp. 3033–3045.
- [11] A. Liu et al., "Contrastive context-aware learning for 3D high-fidelity mask face presentation attack detection," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2497–2507, 2022.
- [12] X. Luo, X. Han, W. Zuo, X. Wu, and W. Liu, "MLaD²: A semi-supervised money laundering detection framework based on decoupling training," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4518–4533, 2024.
- [13] Y. Fang, J. Xie, Y. Zhao, L. Chen, Y. Gao, and K. Zheng, "Temporal-frequency masked autoencoders for time series anomaly detection," in *Proc. IEEE 40th Int. Conf. Data Eng. (ICDE)*, May 2024, pp. 1228–1241.
- [14] S. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
- [15] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1135–1144.
- [16] D. Nemirovsky, N. Thiebaud, Y. Xu, and A. Gupta, "CounterGAN: Generating counterfactuals for real-time recourse and interpretability using residual gans," in *Proc. Uncertainty Artif. Intell.*, 2022, pp. 1488–1497.
- [17] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the gdpr," *Harv. JL Tech.*, vol. 31, p. 841, Apr. 2018.
- [18] S. Nagesh, N. Mishra, Y. Naamad, J. M. Reh, M. Shah, and A. Wagner, "Explaining a machine learning decision to physicians via counterfactuals," in *Proc. Conf. Health*, 2023, pp. 556–577.
- [19] Y. Goyal, Z. Wu, J. Ernst, D. Batra, D. Parikh, and S. Lee, "Counterfactual visual explanations," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 2376–2384.
- [20] E. Ates, B. Aksar, V. J. Leung, and A. K. Coskun, "Counterfactual explanations for multivariate time series," in *Proc. Int. Conf. Appl. Artif. Intell. (ICAPAI)*, May 2021, pp. 1–8.
- [21] E. Delaney, D. Greene, and M. T. Keane, "Instance-based counterfactual explanations for time series classification," in *Proc. Int. Conf. Case-Based Reasoning*, 2021, pp. 32–47.
- [22] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [23] O. Queen, T. Hartvigsen, T. Koker, H. He, T. Tsiligkaridis, and M. Zitnik, "Encoding time-series explanations through self-supervised model behavior consistency," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–31.
- [24] S. Tonekaboni, S. Joshi, K. R. Campbell, D. Duvenaud, and A. Goldenberg, "What went wrong and when? instance-wise feature importance for time-series black-box models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 799–809.

- [25] K. K. Leung, C. Rooke, J. Smith, S. Zuberi, and M. Volkovs, "Temporal dependencies in feature importance for time series prediction," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023, pp. 1–18.
- [26] J. Yan and H. Wang, "Self-interpretable time series prediction with counterfactual explanations," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 39110–39125.
- [27] Z. Li et al., "Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 3220–3230.
- [28] H. Kang and P. Kang, "Transformer-based multivariate time series anomaly detection using inter-variable attention mechanism," *Knowl.-Based Syst.*, vol. 290, Apr. 2024, Art. no. 111507.
- [29] G. E. P. Box and D. A. Pierce, "Distribution of residual autocorrelations in autoregressive-integrated moving average time series models," *J. Amer. Stat. Assoc.*, vol. 65, no. 332, pp. 1509–1526, Dec. 1970.
- [30] N. Kant and M. Mahajan, "Time-series outlier detection using enhanced k-means in combination with PSO algorithm," in *Engineering Vibration, Communication and Information Processing: ICoEVC 2018*, India. Cham, Switzerland: Springer, 2019, pp. 363–373.
- [31] P. Karczmarek, A. Kierszyn, W. Pedrycz, and E. Al, "K-means-based isolation forest," *Knowl.-Based Syst.*, vol. 195, May 2020, Art. no. 105659.
- [32] Y. Yang, H. Zhang, and Y. Li, "Long-distance pipeline safety early warning: A distributed optical fiber sensing semi-supervised learning method," *IEEE Sensors J.*, vol. 21, no. 17, pp. 19453–19461, Sep. 2021.
- [33] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [34] X. Chen and K. He, "Exploring simple Siamese representation learning," in *Proc. IEEE Comput. Soc. Conf. Comput. Vision Pattern Recognit.*, Jun. 2021, pp. 15750–15758.
- [35] J.-B. Grill et al., "Bootstrap your own latent—a new approach to self-supervised learning," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 21271–21284.
- [36] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
- [37] D. Luo and X. Wang, "Moderntcn: A modern pure convolution structure for general time series analysis," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 1–43.
- [38] X. Wu et al., "CATCH: Channel-aware multivariate time series anomaly detection via frequency patching," in *Proc. 13th Int. Conf. Learn. Represent.*, 2025, pp. 1–15.
- [39] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023, pp. 1–24.
- [40] A. Dosovitskiy et al., "An image is worth 16 × 16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–21.
- [41] K. Sohn, X. Yan, and H. Lee, "Learning structured output representation using deep conditional generative models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 3483–3491.
- [42] A. Abdulaal, Z. Liu, and T. Lenczwicki, "Practical approach to asynchronous multivariate time series anomaly detection and localization," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2485–2494.
- [43] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 387–395.
- [44] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ICS security," in *Proc. Int. Workshop Cyber-Phys. Syst. Smart Water Netw. (CySWater)*, Apr. 2016, pp. 31–36.
- [45] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
- [46] K.-H. Lai, D. Zha, J. Xu, Y. Zhao, G. Wang, and X. Hu, "Revisiting time series outlier detection: Definitions and benchmarks," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021, pp. 1–13.
- [47] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [48] B. Zhou, S. Liu, B. Hooi, X. Cheng, and J. Ye, "BeatGAN: Anomalous rhythm detection using adversarially generated time series," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 4433–4439.
- [49] O. D. Anderson, *Time-Series*, 2nd ed. London, U.K.: Charles Griffin, 1976.
- [50] S. Tariq et al., "Detecting anomalies in space using multivariate convolutional LSTM with mixtures of probabilistic PCA," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, Jul. 2019, pp. 2123–2133.
- [51] T. Zhou et al., "One fits all: Power general time series analysis by pretrained lm," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 43322–43355.
- [52] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
- [53] T. Yairi, N. Takeishi, T. Oda, Y. Nakajima, N. Nishimura, and N. Takata, "A data-driven health monitoring method for satellite housekeeping data based on probabilistic clustering and dimensionality reduction," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 53, no. 3, pp. 1384–1401, Jun. 2017.
- [54] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–19.
- [55] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [56] L. Shen, Z. Li, and J. Kwok, "Timeseries anomaly detection using temporal hierarchical one-class network," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 13016–13026.
- [57] Y. Shin et al., "ITAD: Integrative tensor-based anomaly detection system for reducing false positives of satellite systems," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 2733–2740.
- [58] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, Dec. 2008, pp. 413–422.
- [59] R. Prescott Adams and D. J. C. MacKay, "Bayesian online changepoint detection," 2007, *arXiv:0710.3742*.
- [60] M. Perslev, M. H. Jensen, S. Darkner, P. Jennum, and C. Igel, "U-time: A fully convolutional network for time series segmentation applied to sleep staging," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 4415–4426.
- [61] S. Deldari, D. V. Smith, H. Xue, and F. D. Salim, "Time series change point detection with self-supervised contrastive predictive coding," in *Proc. Web Conf.*, Apr. 2021, pp. 3124–3135.
- [62] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 2672–2680.
- [63] A. V. Looveren and J. Klaise, "Interpretable counterfactual explanations guided by prototypes," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, 2021, pp. 650–665.
- [64] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
- [65] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [66] R. K. Mothilal, A. Sharma, and C. Tan, "Explaining machine learning classifiers through diverse counterfactual explanations," in *Proc. Conf. Fairness, Accountability, Transparency*, Jan. 2020, pp. 607–617.
- [67] J. Paparrizos, P. Boniol, T. Palpanas, R. S. Tsay, A. Elmore, and M. J. Franklin, "Volume under the surface: A new accuracy evaluation measure for time-series anomaly detection," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2774–2787, Jul. 2022.
- [68] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
- [69] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 16344–16359.
- [70] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.



Dawei Zhao (Member, IEEE) received the Ph.D. degree in cryptology from Beijing University of Posts and Telecommunications, Beijing, China, in 2014. He is currently a Professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), China. His main research interests include network security, complex networks, and epidemic spreading dynamics.



Haoran Li is currently pursuing the master's degree in computer science and technology with Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), China. His current research interests include anomaly detection and interpretable machine learning.



Zhen Wang (Fellow, IEEE) received the Ph.D. degree from Hong Kong Baptist University, Hong Kong, in 2014. He is a Distinguished Professor and the Dean of the School of Cybersecurity, Northwestern Polytechnical University, Xi'an, China. He has published more than 100 articles in *PNAS*, *Science Advances*, *Nature Communications*, *Physical Review Letters*, *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, *IEEE/ACM TRANSACTIONS ON NETWORKING*, *IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY*, *IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS*, *IEEE TRANSACTIONS ON CYBERNETICS*, *IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING*, *NeurIPS*, *ICLR*, *ICML*, *WWW*, *IJCAI*, and *AAAI* with more than 30 000 citations. His research focuses on artificial intelligence, networks, multiagent systems, and games. He is an Elected Member of Academia Europaea/the Academy of Europe, European Academy of Sciences and Arts, and a fellow of AAAI and IOP.



Lijuan Xu received the master's degree in computer science and technology from Shandong University, Jinan, China, in 2007. She is currently an Associate Professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), China. Her main research interests include network security, industrial internet security, and computer forensics.



Haipeng Peng received the M.S. degree in system engineering from Shenyang University of Technology, Shenyang, China, in 2006, and the Ph.D. degree in signal and information processing from Beijing University of Posts and Telecommunications, Beijing, China, in 2010. He is currently a Professor with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. He has co-authored 200 scientific articles. His research interests include information security, network security, and complex networks.